

Graduate Algorithms

Quiz 2: Solutions

August 17, 2026

Question 1: Master Theorem

3 marks

Proposed by Vardhan Kumar Ray

Consider the recurrence $T(n) = aT(n/b) + f(n)$, where $a > 0$, $b > 1$ and $f(n)$ is asymptotically positive, and let $n^{\log_b a}$ be the critical exponent.

1. If $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some $k \geq 0$, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.
3. If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some $\varepsilon > 0$, and $af(n/b) \leq cf(n)$ for some $c < 1$ and all large n , then $T(n) = \Theta(f(n))$.

For each recurrence below, fill in the table: whether this Master Theorem applies, which case, and the asymptotic solution.

Recurrence	MT applicable? (Y/N)	Case (1/2/3)	Asymptotic solution
$T(n) = 6T(n/3) + n^2 \log n$	Y	3	$\Theta(n^2 \log n)$
$T(n) = 29T(n/29) + n/\log n$	N	N/A	N/A

Solution. First recurrence. Here $a = 6$, $b = 3$, so the critical exponent is $n^{\log_3 6} = n^{1+\log_3 2}$, and $\log_3 6 \approx 1.631$. Take $\varepsilon = 2 - \log_3 6 \approx 0.369 > 0$; then $f(n) = n^2 \log n = \Omega(n^{\log_3 6 + \varepsilon})$, so f is polynomially larger than the critical exponent. The regularity condition holds:

$$6f(n/3) = 6 \cdot \frac{n^2}{9} \log \frac{n}{3} = \frac{2}{3} n^2 \log \frac{n}{3} \leq \frac{2}{3} f(n),$$

so $c = 2/3 < 1$ works. Case 3 gives $T(n) = \Theta(f(n)) = \Theta(n^2 \log n)$.

Second recurrence. Here $a = b = 29$, so the critical exponent is $n^{\log_{29} 29} = n$, and $f(n) = n/\log n$. This f is smaller than n , but not *polynomially* smaller: for every $\varepsilon > 0$ we have $\log n = o(n^\varepsilon)$, hence

$$\frac{n}{\log n} = \omega\left(\frac{n}{n^\varepsilon}\right) = \omega(n^{1-\varepsilon}),$$

so $f(n) \neq O(n^{1-\varepsilon})$ and Case 1 fails. It is also not $\Theta(n \log^k n)$ for any $k \geq 0$, so Case 2 fails, and it is certainly not $\Omega(n^{1+\varepsilon})$. The recurrence falls in the gap below Case 1, and the Master Theorem does not apply.

(The true answer is $\Theta(n \log \log n)$.)

Marking. 1.5 per row, 0.5 per cell.

Question 2: Comparing Growth Rates

4 marks

Proposed by Vardhan Kumar Ray

Each cell below gives a pair of functions f and g . Mark every relation with a tick if it holds for that pair and a cross if it does not. Here O , Ω and Θ abbreviate $f = O(g)$, $f = \Omega(g)$ and $f = \Theta(g)$. The first row is worked out as examples.

$f(n)$	$g(n)$	O	Ω	Θ	$f(n)$	$g(n)$	O	Ω	Θ
n^{29}	$2004^{29} \cdot n$	×	✓	×	29^n	$(29n)!$	✓	×	×
$\log^{29} n$	$n^{0.29}$	✓	×	×	$n^{\log 29}$	$29^{\log n}$	✓	✓	✓
$n^{0.29}$	$(0.29)^n$	×	✓	×	$\log(\sqrt[29]{n})$	$\sqrt[29]{\log n}$	×	✓	×

Solution. (1) $\log^{29} n$ vs. $n^{0.29}$. Any fixed power of $\log n$ is dominated by any positive power of n : substituting $n = 2^m$ turns the comparison into m^{29} with $2^{0.29m}$, and the exponential wins. So $\log^{29} n = o(n^{0.29})$ and only O holds.

(2) $n^{\log 29}$ vs. $29^{\log n}$. These are the *same* function. Taking $\log$ of each, $\log(n^{\log 29}) = \log 29 \cdot \log n = \log(29^{\log n})$, so $n^{\log 29} = 29^{\log n}$ identically, and all three relations hold.

(3) $n^{0.29}$ vs. $(0.29)^n$. The base 0.29 is less than 1, so $(0.29)^n \rightarrow 0$ while $n^{0.29} \rightarrow \infty$. Only Ω holds.

(4) $\log(\sqrt[29]{n})$ vs. $\sqrt[29]{\log n}$. Write $L = \log n$. The left side is $\log(n^{1/29}) = L/29$ and the right side is $L^{1/29}$. Since $L/29$ grows faster than $L^{1/29}$, only Ω holds.

Marking. 1 per (f, g) pair, awarded only if all three entries for that pair are correct; a pair with a missing, extra or wrong mark scores 0 for that pair.

Question 3: Ordering by Growth

3 marks

Proposed by Vardhan Kumar Ray

Arrange the following six functions in increasing order of growth, that is, produce an ordering $g_1, g_2, \dots, g_6$ with $g_{i+1} = \Omega(g_i)$ for each i . If any two of them are Θ of each other, say so explicitly. Write only the ordering, no working is required. Marks are awarded only for a completely correct ordering.

$$n!, \quad (\sqrt{2})^{\log n}, \quad n^2, \quad \log(n!), \quad \log^5 n, \quad (3/2)^n$$

Solution. The ordering is

$$\log^5 n \prec (\sqrt{2})^{\log n} \prec \log(n!) \prec n^2 \prec (3/2)^n \prec n!,$$

where $g_i \prec g_{i+1}$ means $g_{i+1} = \Omega(g_i)$. No two are Θ of each other, so all six classes are distinct.

Two of the entries are:

$$(\sqrt{2})^{\log n} = 2^{\frac{1}{2} \log n} = \sqrt{n}, \quad \log(n!) = \Theta(n \log n).$$

Marking. 3 for a completely correct ordering, 0 otherwise, as stated on the quiz paper.

Question 4: The Four-Item Sorter

2 marks

Proposed by Vardhan Kumar Ray

A piece of hardware performs a single primitive operation: handed any four of the items, it returns those four in sorted order. No other operation on the items is available. Consider algorithms that sort $n \geq 4$ items using this primitive alone. Of the four bounds listed, tick the strongest one that is a valid lower bound on the number of primitive operations used in the worst case.

$$(a) \Omega(n) \quad (b) \Omega(n \log n) \quad (c) \Omega(\log n) \quad (d) \Omega(\sqrt{n})$$

Solution. The answer is (b), $\Omega(n \log n)$.

Run the decision-tree argument with a branching factor larger than 2. A single primitive operation on four items has $4! = 24$ possible outcomes, so an algorithm using t operations in the worst case corresponds to a tree of degree at most 24 and height at most t , which has at most 24^t leaves. Every one of the $n!$ input permutations must reach its own leaf, so

$$24^t \geq n! \quad \implies \quad t \geq \frac{\log(n!)}{\log 24} = \Omega(n \log n),$$

using $\log(n!) = \Theta(n \log n)$. The constant $1/\log 24$ is absorbed by the Ω , so widening the primitive from pairs to 4-tuples does not change the asymptotic bound. Options (a), (c) and (d) are all valid lower bounds too, being weaker, but (b) is the strongest of the four.

Marking. 2 for (b), 0 otherwise.

Question 5: Finding the Maximum

5 marks

Proposed by Vardhan Kumar Ray

We work in the comparison model: an algorithm may only compare two elements and learn which is larger, and it is charged one unit per comparison. Let A be an array of $n \geq 2$ distinct elements.

Prove that every deterministic algorithm that finds the maximum of A must make at least $n - 1$ comparisons in the worst case.

Solution. Say an element *loses* a comparison if it is the smaller of the two compared. Each comparison has exactly one loser, so if the algorithm performs t comparisons on some input, at most t distinct elements ever lose.

Claim. On any input on which the algorithm is correct, every element except the maximum must lose at least one comparison.

Proof of claim. Fix an input A , let m be its maximum, and suppose some $x \neq m$ never loses a comparison during the run. Build a new input A' from A by raising the value of x above every other element, leaving all other values unchanged. Compare the two runs step by step. The only comparisons whose outcome could differ are those involving x ; in the run on A the element x won every one of them, and raising its value keeps it the winner. So every comparison has the same outcome in both runs. The algorithm is deterministic, so its next comparison depends only on the outcomes so far; by induction the two runs are identical and produce the same output, namely m . But the maximum of A' is $x \neq m$, so the algorithm is wrong on A' . Hence no such x exists. $\square$

By the claim, at least $n - 1$ distinct elements lose at least one comparison, and since each comparison produces only one loser, the algorithm makes at least $n - 1$ comparisons. As this holds on every input, it holds in the worst case. $\blacksquare$

Remark. The bound is tight: one scan keeping a running maximum uses exactly $n - 1$ comparisons.

Marking. 1 for the observation that each comparison has exactly one loser. 3 for the claim that every non-maximum element must lose, with a valid argument the exchange argument above, or any equivalent. 1 for combining the two into the $n - 1$ bound.

Question 6: Coarsening Merge Sort

3 marks

Proposed by Vardhan Kumar Ray

Merge sort runs in $\Theta(n \log n)$ worst-case time and insertion sort in $\Theta(n^2)$, but insertion sort has smaller constant factors and so is often faster on short arrays. This suggests *coarsening* the leaves of the recursion: split the input into n/k sublists of length k , sort each one with insertion sort, and then merge them using the usual merging mechanism. Here k is a parameter to be chosen.

You may use, without proof, the fact that the modified algorithm runs in $\Theta(nk + n \log(n/k))$ worst-case time. What is the largest value of k , as a function of n , for which the modified algorithm still matches standard merge sort asymptotically? Justify your answer.

Solution. The answer is $k = \Theta(\log n)$.

For every $k \geq 1$ we have $\log(n/k) \leq \log n$, so $n \log(n/k) = O(n \log n)$.

Suppose $k = \omega(\log n)$. Then $nk = \omega(n \log n)$, and since both terms in $nk + n \log(n/k)$ are non-negative, the total is $\omega(n \log n)$ as well (strictly worse than merge sort). So we need $k = O(\log n)$.

And $k = \Theta(\log n)$ does work. With $k = \Theta(\log n)$ the first term is $nk = \Theta(n \log n)$, and the second is

$$n \log \frac{n}{k} = n (\log n - \log \log n - \Theta(1)) = \Theta(n \log n),$$

because $\log \log n = o(\log n)$. Adding the two gives $\Theta(n \log n)$, matching standard merge sort. Hence the largest admissible choice is $k = \Theta(\log n)$.